TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

WEEKLY REPORT 4

Khảo sát không gian thiết kế shared L2 Cache
đánh giá bằng gem5

Team lead: PhD. Lê Minh Sơn
Mentor: Cao Văn Nam

Họ và tên SV: Choi Won Seok
MSSV: 24200032

Table of content

1. Progress Summary 1
2. Implementation Improvement 1
2.1 Baseline Model Update 1
2.2 Measurement Method Update 2
2.3 Data Parser Update 3
3. CoreMark Workload 3
3.1 Survey of CVA6 Benchmarks 3
3.2 Porting CoreMark to gem5 3
3.3 CoreMark Sweep 4
4. Future Goals 4

Weekly Report

1. Progress Summary

This week, I aligned the simulation model with the team's CVA6 baseline and changed how each simulation is measured. I took Tiến Anh's cva6_baseline.py and copied its values into my own model.py, keeping the existing simple structure, so that the clock, core, L1 caches, crossbar, L2 cache and DRAM now follow the team's baseline instead of the generic placeholder values used in previous weeks.

I also replaced the "run until every program ends" method with the baseline's measurement method: a 2 M-cycle warm-up, a statistics reset, and then exactly 10 M measured cycles. This addresses two limitations from last week's analysis: the initialization phase of a workload was counted in the results, and in mixed-workload runs the cores were not measured over the same window. The parser constants were updated to match the new clock and L2 latency.

Finally, I searched the CVA6 repository for realistic workloads, ported CoreMark to gem5 without modifying CVA6's benchmark sources, and prepared a six-run CoreMark sweep (CoreMark alone and CoreMark on four cores, at 256 KiB, 1 MiB and 4 MiB of L2).

2. Implementation Improvement

2.1 Baseline Model Update

Last week's model used generic parameters: a 1 GHz clock, gem5 MinorCPU defaults, and a 10-cycle L2 hit latency. This week I copied the values from Tiến Anh's cva6_baseline.py into model.py. The structure of model.py is unchanged; only the parameter values were replaced. The changes are summarized below:

Part	Before	Now (baseline)
Clock	1 GHz	100 MHz
Core	MinorCPU defaults	Single-issue, commit width 2, extra decode-to-execute cycle
L1I / L1D	2-cycle hit, LRU	1-cycle hit, Random replacement, 8 targets per MSHR, 8 write buffers, L1I read-only
L1 MSHRs	4	4 (the baseline file specifies 1; see note below)
Crossbar	Defaults	16 B/cycle, 1-cycle latencies
L2	10-cycle hit	2-cycle hit / 1-cycle response, non-inclusive (mostly_incl), 32 MSHRs, 16 write buffers
DRAM	DDR4-2400	Unchanged (DDR4-2400, one channel)

The baseline file specifies 1 MSHR for the L1 caches. I kept 4, the value I had been using, and treated the 1 in the baseline file as an error, since a single MSHR would limit each L1 cache to one outstanding miss. This is a deliberate deviation from the baseline file and should be confirmed with its author.

The core and cache parameters that are not listed above, and the L2 capacity sweep, are unchanged; L2 capacity remains the parameter being explored. One consequence of the new baseline is that cycle-based numbers from earlier reports are no longer comparable with this week's. The DRAM timing is fixed in absolute time, so at 100 MHz a DRAM access costs far fewer core cycles than at 1 GHz (for example, the 75–99 cycle L2 miss latencies in Report 3 were measured at 1 GHz). Miss rates and MPKI remain comparable in shape, but the sweeps need to be re-run under the new baseline before they are compared.

2.2 Measurement Method Update

Previously, each simulation ran until every program had finished. Last week's analysis exposed two limitations of that approach:

- Initialization was counted in the results. For example, the stream workload's miss rate at 4 MiB was lowered by hits in its initialization phase, which does not reflect its steady-state behavior.
- In mixed-workload runs, each core finished at a different time, so the cores were not measured over the same window and interference was only present for part of some cores' execution.

To fix both, I adopted the baseline's method:

- Warm-up: simulate 2 M cycles so that initialization and cold misses fall outside the measurement.
- Reset statistics at the end of the warm-up.
- Measure: simulate exactly 10 M cycles, so all four cores are measured over the same window.

Because every run now covers the same number of cycles, simulated time is identical across runs by construction and can no longer be used to compare configurations. Performance comparisons must instead use the number of instructions committed within the window (or IPC), together with the per-instruction metrics already in the pipeline (miss rate, MPKI and AMAT). Workloads must also run longer than the 12 M cycles of warm-up plus measurement, otherwise they end before the window closes.

2.3 Data Parser Update

The parser contains two constants that depend on the model, and both were updated to match the new baseline: TICKS_PER_CYCLE changed to 10000 (a 100 MHz clock has a 10 ns period, which is 10,000 ticks at 1 ps per tick), and L2_HIT_CYCLES changed to 2. These constants are used to convert simulation ticks into cycles for the latency and AMAT columns, so leaving them at the old values would have silently produced wrong results.

3. CoreMark Workload

3.1 Survey of CVA6 Benchmarks

The synthetic workloads used so far (loop, temporal, stream and graph) were designed to stress specific access patterns, but they are not representative of what the CVA6 will actually run. I therefore looked for realistic workloads in the CVA6 repository and found CoreMark, Dhrystone, and the riscv-tests kernels (vvadd, mm, spmv, qsort, median, and others). Two findings limit how they can be used:

- **They are bare-metal programs.** They carry their own start-up code, use a serial port at a fixed address and read hardware counters directly, so they cannot run in gem5's Linux mode as they are.
- **They are tiny.** CoreMark uses about 2 KB of data, which fits easily in the 32 KiB L1 data cache. These benchmarks measure the core, not the memory system.

Because of the second point, I expect these benchmarks to say little about L2 sizing on their own. They are still useful as a realistic reference point for the control-type workload the CVA6 is designed to run.

3.2 Porting CoreMark to gem5

I chose CoreMark for the port. CVA6's benchmark files were kept unchanged and only the platform layer was replaced:

- core_portme.c and core_portme.h were rewritten to use the standard printf for output and clock() instead of the hardware cycle counter.
- uart.c was dropped, since there is no serial port to drive.
- The build uses CVA6's own settings (-O3 and PERFORMANCE_RUN), except that ITERATIONS is set to 1000 so that the program keeps running beyond the 12 M-cycle warm-up and measurement window.

To verify the port, I checked that CoreMark's built-in self-check passes and reports the standard checksums, which indicates that the benchmark executes correctly under gem5.

3.3 CoreMark Sweep

The sweep consists of six runs: CoreMark alone (one core active, three idle) and CoreMark on all four cores, each at an L2 capacity of 256 KiB, 1 MiB and 4 MiB. All runs use the new baseline and the 2 M warm-up / 10 M measurement method. The measured results are shown below:

Workload	Metric	256 KiB	1 MiB	4 MiB
coremark	L2 miss rate	TBD	TBD	TBD
coremark	L2 MPKI	TBD	TBD	TBD
coremark x4	L2 miss rate	TBD	TBD	TBD
coremark x4	L2 MPKI (core 0)	TBD	TBD	TBD

[Fill in the table above from the CoreMark sweep results, and confirm or correct the expectation below.]

The expectation is a flat curve with very low L2 MPKI, because CoreMark's roughly 2 KB of data stays in the L1 caches and the L2 sees little traffic once the warm-up is over. If the results confirm this, the conclusion is that a typical CVA6 control workload fits in the L1 caches, so L2 sizing cannot be justified by this kind of workload. It has to be driven by data-heavy (DSP and vector) traffic, which is where the synthetic workloads of the previous weeks (stream, graph, temporal and loop) remain the more informative tools.

4. Future Goals

In the upcoming weeks, I plan to re-run the synthetic workload sweeps (loop, temporal, stream and graph) under the new baseline and measurement method, making sure each workload runs longer than the 12 M-cycle window, so that the results are comparable with the CoreMark runs. I also plan to look for or build data-heavy workloads that better represent DSP and vector traffic, since those are expected to drive the L2 sizing decision. After that, I will continue exploring replacement policies, prefetchers and indexing functions, and study TCM in more depth, as planned in the previous report.